

Table des matières

Arbre binaire ordonné	2
Définition	2
Propriétés	3
Recherche dans un arbre binaire de recherche	4
Version itérative de la recherche dans un arbre binaire de recherche	7
Exemple : construction d'un ABR	8
Insertion dans un arbre binaire ordonné	9
Problème.....	9
Méthode 1 : insertion en feuille.....	9
Méthode 2 itération d'insertion en feuille	11
Principe	11
Etapas de l'algorithme.....	11
Recherche de la Place d'insertion.....	13
Schéma récursif de la Recherche de la Place d'insertion.....	14
Algorithme réalisant l'insertion.....	16
Application : tri d'un tableau	17
Principe	17
Algorithme de tri	19
Recherche du plus petit(resp. Plus Grand) élément dans un ABR	21
Principe.....	21
Recherche l'adresse du plus petit (plus grand) élément dans un ABR	23
Algorithmes de suppressions	24
Suppression du nœud racine	24
Hypothèse : non estVide(a).....	24
Principe : Suppression de la racine on distingue les cas suivants :.....	24
Suppression d'un élément dans un arbre binaire ordonné	27
Problème.....	27
Exemple.....	28
Exercice	30
Intérêts des ABR	31

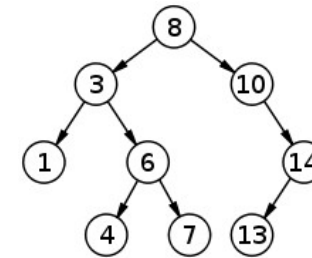
Arbre binaire ordonné

Arbre binaire ordonné = arbre binaire de recherche

Soit ($\leq$) une relation d'ordre sur l'ensemble des valeurs (données) attachées aux nœuds d'un arbre binaire.

Définition

Un arbre binaire est dit ordonné si la chaîne infixée des valeurs correspondant au parcours infixe est ordonnée.

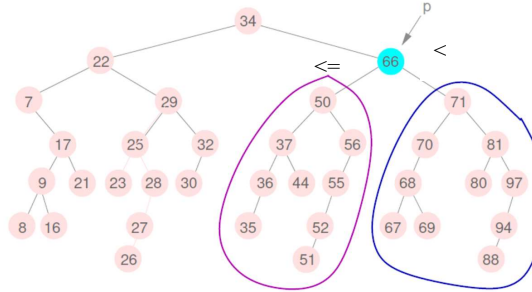
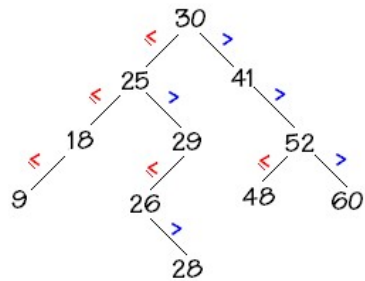
**Rappel du Parcours Infixe**

Ce parcours consiste à effectuer dans l'ordre :

- Parcours du fils gauche
- Traitement de la racine
- Parcours du fils droit

Le parcours infixe de l'arbre donne une liste ordonnée de valeurs :

[1, 3, 4, 6, 7, 8, 10, 13, 14]



Propriétés

Un arbre binaire de recherche A est un arbre qui possède les propriétés suivantes :

Si e est la donnée associée au nœud A, alors toutes les données associées au FilsGauche(A) sont inférieures ou égale à e et tous les données associées au FilsDroit(A) sont supérieurs à e.

Autrement dit : chaque nœud de l'arbre A vérifié les propriétés suivantes :

Si les deux fils du nœud A sont non vides :

$$\text{donnee}(\text{filsGauche}(A)) \leq \text{donnee}(A) < \text{donnee}(\text{filsDroit}(A))$$

Si le fils gauche est vide : $\text{donnee}(A) < \text{donnee}(\text{filsDroit}(A))$

Si le fils droit est vide : $\text{donnee}(\text{filsGauche}(A)) \leq \text{donnee}(A)$

Remarque

Les deux sous arbres gauches et droit sont ordonnés.

Recherche dans un arbre binaire de recherche

Fonction $\text{rechEltABR}(\text{TElement} \times \text{Arbre}) \rightarrow \text{Booléenne}$

Fonction $\text{rechEltABR}(\text{elt}, a) \rightarrow \text{booléenne}$

Raisonnement

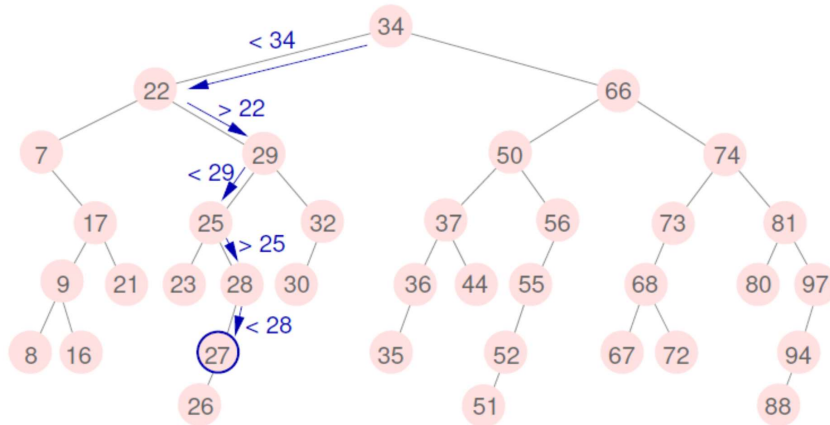
Si l'arbre

La recherche est terminée. La fonction retourne faux

Si l'arbre n'est pas vide, on distingue trois cas :

- Soit la donnée de la racine de l'arbre est égale à l'élément recherché auquel cas la recherche est terminée. La fonction retourne vraie.
- Soit la donnée de la racine est inférieur à l'élément recherché auquel cas on effectue la recherche par rapport au fils droit.
- Soit la donnée de la racine est Supérieur à l'élément recherché auquel cas on effectue la recherche par rapport au fils gauche.

Exemple



recherche de la valeur 27

Fonction rechEltABR(elt, a) → Booléenne

P.F elt : type TElement(E). a : type arbre (E).

Début

Si estVideArbre(a) alors

Renvoyer faux

Sinon

Si donnee(a)=elt alors

Renvoyer Vrai

Sinon

Si elt < donnee(a)

Renvoyer rechEltArbreOrdonné(elt, filsGauche(a))

sinon

Renvoyer rechEltArbreOrdonné(elt, filsDroit(a))

finsi

finsi

finsi

FIN

Version itérative de la recherche dans un arbre binaire de recherche

Fonction itRechEltABR(elt, a) → Booléenne

P.F elt : type TElement(E). a : type Arbre (E).

Début

Tantque non estVide(a) && donnee(a) ≠ elt faire

Si elt < donnee(a)

a ← filsGauche(a)

sinon

a ← filsDroit(a)

finsi

FinTq

Renvoyer non estVide(a)

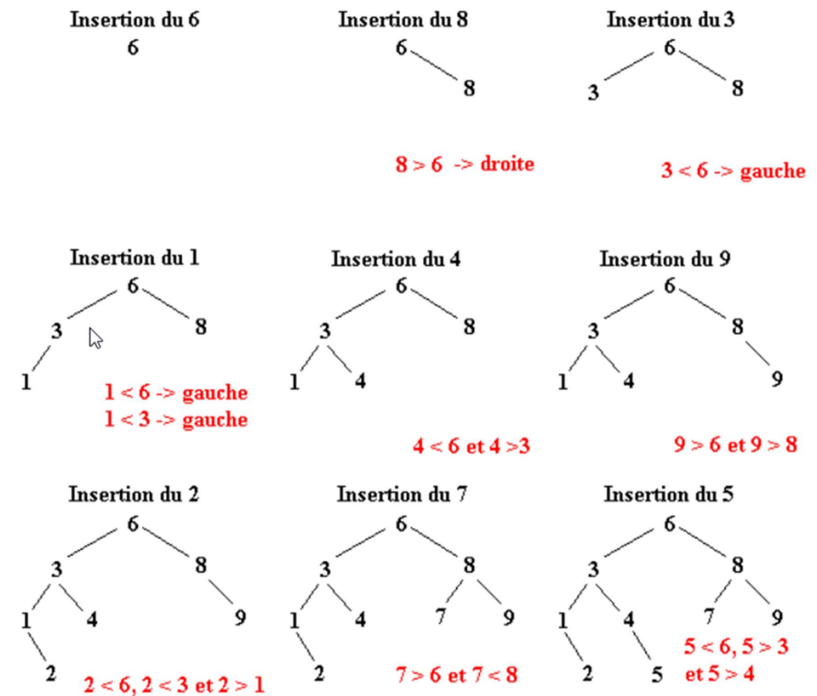
FIN

Exemple : construction d'un ABR

Construction d'un arbre binaire de recherche à partir de la liste d'entiers :

[6, 8, 3, 1, 4, 9, 2, 7, 5]

On définit la construction d'un arbre binaire de recherche par l'insertion successive en respectant la propriété d'ABR.



Insertion dans un arbre binaire

ordonné

Problème

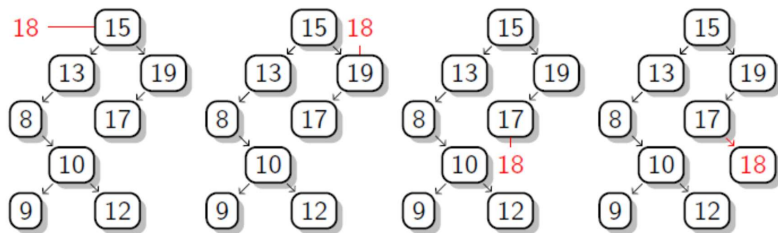
On souhaite insérer un élément donné dans un arbre ordonné tel qu'il demeure ordonné.

Fonction inserABR : TElement x Arbre → Arbre

Méthode 1 : insertion en feuille

Si l'arbre est vide, il suffit de créer une feuille contenant l'élément à insérer. Cette feuille deviendra la racine de l'arbre.

Sinon, on distingue deux cas :



- Soit la donnée de la racine est inférieure à l'élément recherché auquel cas on effectue l'insertion par rapport au fils droit.
- Soit la donnée de la racine est supérieure ou égale à l'élément recherché auquel cas on effectue l'insertion par rapport au fils gauche.

Procédure inserABR(elt , racine)

PF elt : TElement (E) et racine : Arbre (E/S)

Début

Si estVide(racine) alors

racine ← creatFeuille(elt)

Sinon

Si elt > donnee(racine) alors

inserABR(elt, filsDroit(racine))

Sinon // elt <= donnee(racine)

inserABR(elt, filsGauche(racine))

Finsi

Finsi

Fin

Attention : traduction en c

Méthode 2 itération d'insertion en feuille

Principe

Si l'arbre est vide, il suffit de créer une feuille contenant l'élément à insérer.

Sinon, on parcourt l'arbre, afin de rechercher un nœud qui sera le père de l'élément que nous désirons insérer.

Une fois le nœud père trouvé, il suffit de créer une feuille contenant l'élément à insérer, et l'attacher du bon côté du nœud père.

Etapas de l'algorithme

- 1) Trouver la place d'insertion
- 2) Insérer l'élément :
 - Création d'une feuille et remplissage
 - Attachement (réalisation des chaînages)

// Version itérative d'insertion d'un élément dans un arbre binaire ordonné

Procédure it_InserABR(elt, a)

P.F elt : TElement (E). a : Arbre (E/S)

Début

Si estVide(a) alors

 a ← creatFeuille(elt)

Sinon

 // Déterminer l'adresse d'insertion

 adrPere ← adrInserABR(elt, a)

 // On réalise l'insertion

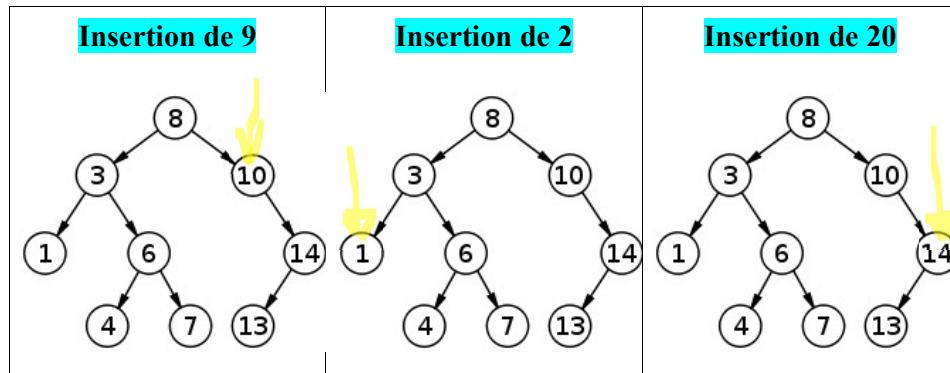
 insertion(elt, AdrPere)

fin

Fin

Recherche de la Place d'insertion

Exemples



On recherche un nœud de l'arbre, qui sera le père de l'élément que nous désirons insérer.

Quand nous l'aurons inséré ; ce nœud **père** est tel que :

$donnee(père) < elt$ et $estVideArbre(FilsDroit(père))$

Ou $donnee(père) \geq elt$ et $estVideArbre(FilsGauche(père))$

Ce nœud père est toujours au bout de la filiation. C'est soit une feuille soit un nœud qui n'a qu'un seul fils.

Schéma récursif de la Recherche de la Place d'insertion

On suppose que l'arbre n'est pas vide.

On distingue deux cas :

- Soit l'élément à insérer est inférieur ou égal ($\leq$) à la donnée de la racine de l'arbre $\rightarrow$ dans ce cas, l'élément fait donc partie du sous arbre gauche, on a encore deux cas :
 - Si le fils gauche est vide, on a trouvé donc le nœud qui sera le père de l'élément à insérer.
 - Si le fils gauche n'est pas vide, la place recherchée en effectuant un appel récursif sur le fils gauche.
- Soit l'élément à insérer est supérieur ($>$) à la donnée de la racine de l'arbre $\rightarrow$ dans ce cas, l'élément fait donc partie du sous arbre droit, on a encore deux cas :
 - Si le fils droit est vide, on a trouvé donc le nœud qui sera le père de l'élément à insérer.
 - Si le fils droit n'est pas vide, la place recherchée en effectuant un appel récursif sur le fils droit.

Fonction adrInserABR : TElement x Arbre → Arbre

Fonction adrInserABR (elt, a) → Arbre

P.F elt : type Telement (E)

a ; type Arbre (E)

Début

Si elt <= donnee(a) alors

 Si estVideArbre(filsGauche(a)) alors

 Renvoyer a

 Sinon

 Renvoyer adrInserABR(elt, filsGauche(a))

 Finsi

Sinon // elt > donnee(a)

 Si estVideArbre(filsDroit(a)) alors

 Renvoyer a

 Sinon

 Renvoyer adrInserABR(elt, filsDroit(a))

 Finsi

Finsi

Fin

Algorithme réalisant l'insertion

Fonction insertion TEelement x Arbre → Arbre

Procédure insert(elt, adrPere)

P.F elt : TEelement (E)

adrPere : Arbre(E)

Debut

f ← creatFeuille(elt)

Si elt <= donnee(adrPere)

 adrPere.filsG ← f

Sinon

 adrPere.filsD ← f

Finsi

Fin

Application : tri d'un tableau

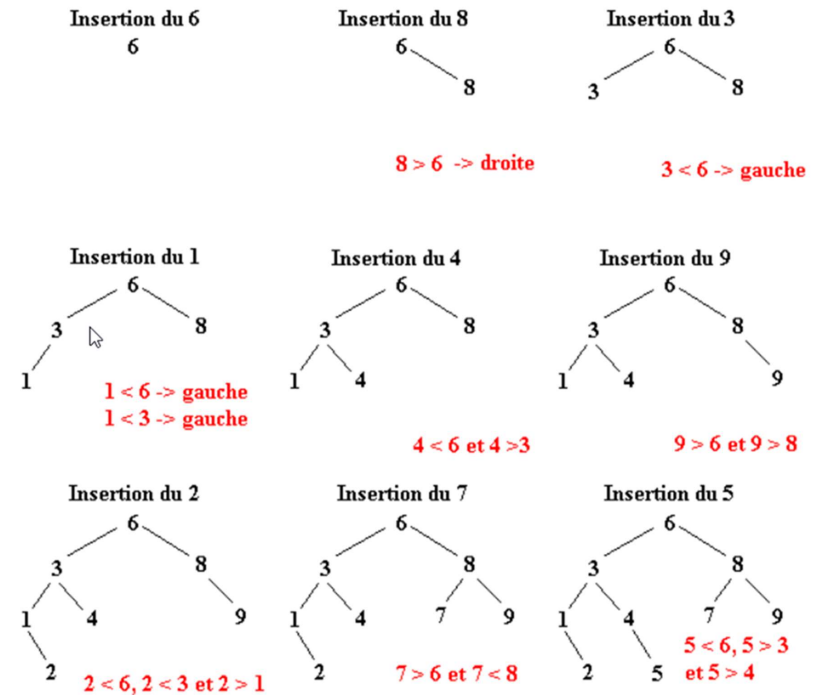
Principe

Pour trier un tableau, on procède comme suit :

- 1) Création d'un arbre binaire ordonné contenant toutes les valeurs du tableau.
- 2) Effectuer un parcours infixe de l'arbre en rangeant les éléments de l'arbre dans le tableau afin d'obtenir un tableau trié.
- 3) Libération de la mémoire occupée par l'arbre binaire ordonné

Exemple : tri du tab = [6, 8, 3, 1, 4, 9, 2, 7, 5]

- 1) Construction d'un arbre binaire de recherche à partir de la liste d'entiers :
On définit la construction d'un arbre binaire de recherche par l'insertion successive en respectant la propriété d'ABR.



- 2) Effectuer un parcours infixe de l'arbre en rangeant les éléments de l'arbre dans le tableau afin d'obtenir un tableau trié.

Tab=[1, 2, 3, 4, 5, 6, 7, 9]

- 3) Suppression de l'ABR

Algorithme de tri

Fonction triParABR : Entier x Tableau $\rightarrow$ Tableau
procédure triParABR(n, tab)

PF n : Entier (E)

tab ; Tableau (E/S)

Début

// Création de l'ABR

racine $\leftarrow$ creatFeuille(tab[1])

Pour i variant de 2 à n faire

InserABR(tab[i], racine)

Fpour

// Parcours Infixe de l'ABR

n $\leftarrow$ 1

parcoursInfixeABRTab(n, racine, tab)

// Libération de la mémoire occupée par l'arbre binaire ordonné

suppressionArbre(racine)

Fin

Rappel Parcours Infixe

Ce parcours consiste à effectuer dans l'ordre :

- a) Parcours du fils gauche
- b) Traitement de la racine
- c) Parcours du fils droit

Procédure parcoursInfixeABRTab(n, racine, tab)

PF n : Entier (E/S)

racine : Arbre (E)

tab : Tableau (E/S)

Début

Si non estVideArbre(racine) alors

parcoursInfixeABRTab(n, filsGauche(racine), tab)

// Traitement de la racine

tab[n] $\leftarrow$ donnee(racine)

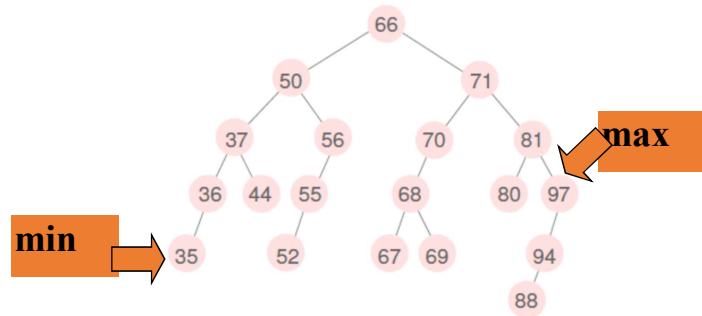
n $\leftarrow$ n+1

parcoursInfixeABRTab(n, filsDroit(racine), tab)

finsi

Fin

Recherche du plus petit(resp. Plus Grand) élément dans un ABR



Principe

Le **plus petit élément** dans un ABR est l'élément le plus à gauche. C'est-à-dire le nœud le plus à gauche qui ne possède pas de fils gauche

Le **plus grand élément** dans un ABR est l'élément le plus à droite. C'est-à-dire le nœud le plus à droite qui ne possède pas de fils droit.

Précondition : L'arbre est non vide

Principe :

Rechercher l'élément le plus à gauche

Fonction minABR : Arbre → TElement

Fonction minABR(racine) --> TElement

P.F racine : Arbre (E)

Début

fg <-- filsGauche(racine)

si estVide(fg) alors

renvoyer donnee(racine)

sinon

renvoyer minABR(fg)

fsi

FIN

Recherche l'adresse du plus petit

(plus grand) élément dans un ABR

Précondition : L'arbre est non vide

Principe :

Rechercher le nœud le plus à gauche

Fonction minABR : Arbre → Arbre

Fonction minABR(racine) --> Arbre

Pré- condition : L'arbre est non vide

P.F racine : Arbre (E)

Début

fg <-- filsGauche(racine)

si estVide(fg) alors

renvoyer racine

sinon

renvoyer minABR(fg)

fsi

Fin

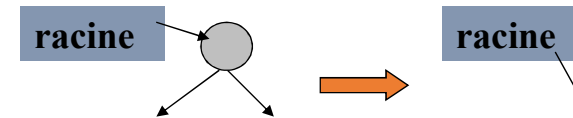
Algorithmes de suppressions

Suppression du nœud racine

Hypothèse : non estVide(a)

Principe : Suppression de la racine on distingue les cas suivants :

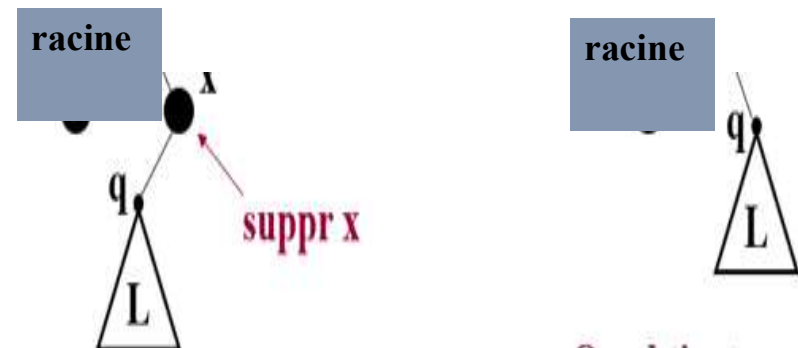
a) Si l'arbre est réduit à une feuille, on supprime la feuille.



b) Sinon, on remplace la racine par son fils droit s'il n'est pas vide et on rattache le fils gauche de la racine au plus petit élément de son fils droit.



Par contre on remplace la racine par son fils gauche si le fils droit de l'arbre est vide.



Version1**Hypothèse : non estVide(a)****Fonction SuppRacineABR : Arbre → Arbre****Procédure SuppRacineABR(racine)**

PF racine : Arbre (E/S)

Début

//Préservation du nœud à supprimer

rr ← racine

// Modification des chaînages

si estFeuille(racine) alors

racine ← initArbre()

else

fg ← filsGauche(racine)

fd ← filsDroit(racine)

Si non estVideArbre(fd) alors

//Recherche du Plus Petit Nœud de FD

ppndfd ← fd

Tantque non estVide(filsGauche(ppndfd))

faire

ppndfg ← filsGauche(ppndfd)

Fintq

ppndfd.filsg ← fg

racine ← fd

Sinon

racine ← fg

Finsi

Finsi

//libération Mémoire

LibMémoire(rr)

Fin**Version2****Procédure SuppRacineABR(racine)**

PF racine : Arbre (E/S)

Début

// Préservation du nœud racine

rr ← racine

// Modification chaînage

si estFeuille(racine) alors

racine ← initArbre()

sinon

fg ← filsGauche(racine)

fd ← filsDroit(racine)

Si non estVideArbre(fd) alors

//Recherche du Plus Petit Nœud de FD

ppndfd ← minABR(fd)

ppndfd.filsg ← fg

Racine ← fd

Sinon

racine ← fg

Finsi

Finsi

// Libération Mémoire

LibMémoire(rr)

Fin

Suppression d'un élément dans un arbre binaire ordonné

Fonction `suppEltABR` : TElement x Arbre $\rightarrow$ Arbre

Précondition : non estVideArbre(a)

Problème

On se propose de supprimer un élément dans un arbre binaire ordonné de telle sorte que l'arbre reste ordonné.

Méthode 1 : Même principe que la recherche dichotomique

Si l'arbre est vide $\rightarrow$ c'est que l'élément à supprimer n'appartient pas à l'ABR

Arrêt de l'algorithme

Si l'arbre est non vide : on distingue les cas suivants :

- Cas où `donnee(a) = elt` $\rightarrow$ on applique la suppression du nœud racine
- Cas où `elt < donnee(a)` $\rightarrow$ on effectue la suppression par rapport au fils gauche de l'ABR.
- Cas où `elt > donnee(a)` $\rightarrow$ on effectue la suppression par rapport au fils droit de l'ABR.

Exemple

Fonction `suppEltABR` : `TElement x Arbre → Arbre`

Procédure `suppEltABR(elt, racine)`

P.F

elt : **TElement** (E) **racine** : **Arbre** (E/S)

Début

Si non `estVideArbre(racine)` alors

 Si `donnee(racine)=elt` alors

`SuppRacineABR(racine)`

 Sinon

 Si `elt < donnee(racine)` alors

`suppEltABR(elt, filsGauche(racine))`

 Sinon

`suppEltABR(elt, filsDroit(racine))`

 Finsi

 Finsi

Finsi

Fin **Attention traduction en C**

Exercice

Vérifier si un arbre binaire donné est un arbre binaire de recherche

Rappel : Propriété d'un ABR

Un ABR est un arbre binaire $a = [racine, filsG, filsD]$ tel que :

Les deux sous arbres `filsG` et `filsD` sont des ABRs

Et

si `filsG` et `filsD` sont non vide alors

$donnee(filsG) \leq donnee(racine) < donnee(filsD)$

si `filsG` est Vide et `filsD` non vide alors

$donne(racine) < donnee(filsD)$

si `filsD` est Vide et `filsG` non vide alors

$donnee(filsG) \leq donne(racine)$

Intérêts des ABR

Les Arbres Binaires de Recherche sont une structure de données qui permet de représenter correctement un ensemble ordonné de données.

Les alternatives (tableaux ou listes) ont de bien moins bons comportements dans les opérations usuelles : ajout / suppression / recherche.

Tous les algorithmes des ABRs, le temps de calcul est proportionnel à la hauteur de l'arbre (et non au nombre de données comme pour les autres alternatives)

Toutefois, dans le pire des cas, la hauteur de l'arbre est le nombre de données dans l'arbre.

En moyenne, les ABRs sont quand même souvent intéressants.

Pour éviter les mauvais cas : **équilibrage des arbres**.